perf_analyzer 使用说明书

Oct 10, 2026 · @john.he

1. 简介

`perf_analyzer` 是一个运行在主机（Ubuntu）上的命令行工具，读取 perf 采集并解析好符号的数据，自动输出一份中文分析报告：数据是否可信、CPU 花在哪些线程和函数上、命中了哪些常见性能问题模式、优先优化什么。它把人工看火焰图的分析思路固化成了规则，适合对 E3 板子上的 `ApplicationServices` 等程序反复做性能分析。

项目	说明
运行环境	主机 Linux（已在 Ubuntu 22.04 验证），单文件 C++17，无第三方依赖
输入	`perf script` 的文本输出（推荐），或 `stackcollapse-perf.pl` 生成的折叠栈，格式自动识别
输出	Markdown 格式的分析报告；可选输出折叠栈，直接交给 `flamegraph.pl` 生成火焰图
配套文件	`perf_analyzer.cpp`（源码）、`Makefile`（编译和一键生成报告）

报告包含七部分：数据质量检查、整体概况、线程分布与最热调用路径、自身/累计耗时排行、规则检测结果、优化优先级、说明。

2. 安装与编译

环境要求：主机上有 `g++`（支持 C++17）、`make`、`perf`；生成火焰图还需要 FlameGraph 脚本（`git clone https://github.com/brendangregg/FlameGraph.git`）。

源码结构（全部放在同一目录，例如 `/home/hejr/test/perfAnalyzer`）：

文件	内容
`main.cpp`	程序入口，串联各模块

文件	内容
`options.h/.cpp`	命令行选项和帮助信息
`input_parser.h/.cpp`	`InputParser` 类：格式识别，解析 perf script 和折叠栈
`dataset.h/.cpp`	`Interner`（函数名字符串池）、`Stack`、`Dataset`（含输出折叠栈）
`elf_symbolizer.h/.cpp`	`ElfFile`、`Symbolizer` 类：读取 ELF 符号表，把运行时地址解析成函数名或链接地址
`addr_resolver.h/.cpp`	地址清单换算：把 `--addr-list` 导出的地址转换成函数名
`symbol_rules.h/.cpp`	函数分类规则：内核/库/业务、系统调用入口、栈底异常帧、可疑 libc 函数
`analyzer.h/.cpp`	`Analyzer` 类：统计、报告输出、规则检测、输出地址清单
`string_utils.h/.cpp`	字符串和格式化工具
`Makefile`	编译、交叉编译、安装、一键生成报告

旧的单文件 `perf_analyzer.cpp` 不再使用，请从目录中删除。

```
cd /home/hejr/test/perfAnalyzer
make -j4        # 编译，生成 ./perf_analyzer（指向 build/perf_analyzer）
make debug      # 调试版本：-O0 -g，带地址和未定义行为检查
make install    # 安装到 /usr/local/bin（需要 root）
make clean      # 清理 build/ 和 perf_analyzer，不会删除源码
```

Makefile 按文件分别编译并自动跟踪头文件依赖，修改某个 `.h` 后只重新编译相关的 `.cpp`。

注意

- Makefile 不继承环境变量中的 `CROSS_COMPILE`，即使 shell 里 source 过 SDK 环境脚本，`make` 仍使用主机 `g++`。
- 需要放到 E3 开发板上运行时，按第 7 节交叉编译和移植。
- 用 root 执行时 `~` 和 `$HOME` 指向 `/root`，hejr 用户目录下的文件要写完整路径（如 `/home/hejr/perfwork`）。

3. 快速开始

三步得到报告和火焰图。前提：板子上运行的程序与主机符号目录（`/home/hejr/symfs_e3`）是同一次编译的产物，已用 md5 核对过。

1. **板子上采集**（在业务繁忙或问题复现时）

```
echo 0  > /proc/sys/kernel/kptr_restrict
echo -1 > /proc/sys/kernel/perf_event_paranoid
cd /tmp
perf record -e cpu-clock -F 199 -m 64 --call-graph fp --proc-map-timeout
10000 \
    -p $(pidof ApplicationServices) -o perf.data -- sleep 120
cat /proc/kallsyms > kallsyms
```

2. **拷贝到主机**

```
scp root@10.166.64.107:/tmp/{perf.data,kallsyms} /home/hejr/perfwork/
```

3. **一键生成报告和火焰图**

```
make -C /home/hejr/test/perfAnalyzer report \
    DATA=/home/hejr/perfwork/perf.data \
    KALLSYMS=/home/hejr/perfwork/kallsyms \
    SYMFS=/home/hejr/symfs_e3 \
    FLAMEGRAPH=/home/hejr/FlameGraph \
    DURATION=120
```

结果在 `/home/hejr/test/perfAnalyzer/report/`：`report.md` 分析报告、`flame.svg` 火焰图，另有中间文件 `out.perf`、`out.folded` 和 perf 解析日志 `perf_script.log`。`DURATION` 填第 1 步 `sleep` 的秒数。

4. 详细用法

4.1 板子端采集要点

- 用 `cpu-clock` 软件事件，比硬件 `cycles` 可靠；`-F 199` 配合 120 秒，样本数能到几千个。只看用户态时改成 `cpu-clock:u`。
- `--proc-map-timeout 10000`：用 `-p` 附加到运行中的进程时给读取内存映射留足时间，否则样本会全部归到 `[JIT]`。

- 导出 `kallsyms` 前必须设置 `kptr_restrict=0`，且驱动模块仍处于加载状态，否则内核函数显示为 `[unknown]`。
- 板子上的 perf 读不到 build-id，用 `md5sum` 核对程序版本：主机 `rootfs_arm64-short/rootfs_partition/` 下的文件应与板子上的完全一致。

4.2 主机端手动分析

```
cd /home/hejr/perfwork
perf script --force -i perf.data --symfs=/home/hejr/symfs_e3 --
kallsyms=kallsyms > out.perf
/home/hejr/test/perfAnalyzer/perf_analyzer out.perf --duration 120 >
report.md
```

`--force` 在 root 读取 hejr 用户的 `perf.data` 时需要；用 hejr 用户执行可以省略。只想看某个线程：加 `--comm DataServices`。

4.3 命令行参数

参数	说明	默认值
`<输入文件>`	`perf script` 输出或折叠栈文件（`--resolve-addrs` 模式下不需要）	必填
`--format auto|script|folded`	输入格式	`auto`（自动识别）
`--duration <秒>`	采集时长，用于计算平均 CPU 占用	不计算
`--freq <Hz>`	采样频率，输入中没有 period 信息时用于估算 CPU 时间	—
`--top <N>`	热点排行条数	15
`--min-pct <百分比>`	规则检测的最小占比阈值	1.0
`--comm <线程名>`	只分析指定线程	全部
`--keep-root-anomalies`	保留栈底的 `[perf-N.map]` / `[JIT]` / `[unknown]` 帧	自动去除
`--folded-out <文件>`	同时输出折叠栈，可直接交给 `flamegraph.pl`	不输出
`--mark-kernel`	折叠栈中内核帧加 `_[k]` 后缀	不加

参数	说明	默认值
`--symfs <目录>`	ELF 文件路径前缀，用于读取未 strip 的程序解析函数名	直接读原路径
`--maps <文件>`	`/proc/<PID>/maps` 格式的内存映射文件，`perf script` 没有加 `--show-mmap-events` 时使用	—
`--no-elf`	关闭 perf_analyzer 自带的 ELF 符号解析	开启
`--addr`	无法解析函数名时显示"文件名+0x链接地址"	只显示库名
`--addr-list <文件>`	输出地址清单（自动启用 `--addr`）	不输出
`--resolve-addrs <文件>`	把地址清单换算成函数名，配合 `--symfs` 使用	—

perf_analyzer 自带 ELF 符号解析：perf 解析不出的用户态地址，会根据 `--show-mmap-events` 输出的内存映射换算成文件内地址，再查 ELF 符号表（`.symtab`，没有则用 `.dynsym`）得到函数名，不依赖 libelf。

4.4 Makefile 目标与变量

目标	作用
`make` / `make release`	编译
`make debug`	编译调试版本
`make report`	perf script 解析 → 分析报告 → 火焰图
`make install` / `make uninstall`	安装到 / 卸载自 `$(PREFIX)/bin`
`make clean`	清理编译产物
`make help`	显示帮助

变量（用于 `make report`）	说明	默认值
`DATA`	perf.data 路径	`perf.data`
`KALLSYMS`	板子导出的 kallsyms 路径	`kallsyms`

变量（用于 `make report`）	说明	默认值
`SYMFS`	符号目录	`$(HOME)/symfs_e3`
`DURATION`	采集秒数	0（不计算 CPU 占用）
`FLAMEGRAPH`	FlameGraph 脚本目录	`$(HOME)/FlameGraph`
`TITLE`	火焰图标题	`ApplicationServices`
`OUT_DIR`	输出目录	`report`
`PERF`	perf 命令	`perf`
`PERF_SCRIPT_FLAGS`	传给 perf script 的额外参数	`--force`
`PREFIX`	安装路径前缀	`/usr/local`

5. 报告解读

先看第一部分判断数据是否可信，再看第五、六部分决定优化方向；所有占比都相对于全部样本。

章节	内容	怎么看
一、数据质量检查	样本数、只有库名的比例、libc 版本不匹配、栈底异常帧、回溯断裂、ELF 符号解析和地址模式统计	出现"libc 版本不一致"或样本数很少时，后面的结论要打折扣，先按提示修正
二、整体概况	样本数、CPU 时间、平均 CPU 占用、内核架构（推断）、内核和系统调用占比、系统调用分布	平均占用很低时，热点只是相对比例；内核架构用于确认 E3（64 位 ARM）还是 SS（32 位 ARM）
三、线程分布	各线程占比、每个线程累计占比最高的业务函数、线程入口函数、主要线程的最热调用路径	先找最耗 CPU 的线程，再顺着调用路径往下看；列出的"线程入口函数"不是热点，已自动跳过
四、热点排行	自身耗时和累计耗时（只列业务函数），类型标注内核、库、业务、地址	自身耗时高说明函数本身慢；累计高但自身低说明耗时在它调用的函数里
五、规则检测结果	命中的问题模式，附调用者、所在线程和建议	调用者是具体的优化落点；调用者是地址时，用第 8 节的方法换算成函数名

章节	内容	怎么看
六、优化优先级	按"占比 ÷ 改动难度"排序	从上往下依次处理
七、说明	统计口径和局限	—

规则检测说明

规则	识别依据	含义与建议
频繁调用 gettid	`gettid`	每次都是系统调用；在线程入口获取一次并缓存
频繁获取系统时间	`clock_gettime`、`gettimeofday`	64 位内核上的 32 位程序常走系统调用；一个周期只取一次
64 位软件除法	`__udivmoddi4`、`__aeabi_uldivmod` 等	32 位 ARM 没有 64 位除法指令；常见于时间换算，减少除法次数
疑似反复排序	名字含 compare/cmp 的业务函数，且由库函数回调	每周期对整个列表 qsort；改用最小堆或有序插入
字符串键 map 查找	字符串键的 `_Rb_tree` / `_Hashtable`，以及 `memcmp`、`strlen`、`std::string` 构造	按名字查属性；初始化时解析成索引或指针缓存
等待类调用占用 CPU	`select`、`poll`、`epoll_wait`、`nanosleep` 等	线程被频繁唤醒，检查超时是否过短
内存分配频繁	`malloc`、`free`、`operator new` 等	减少临时对象和拷贝，用 reserve、复用、move
锁和信号量	`sem_trywait`、`pthread_mutex_*`、`PosixSemTake` 等	检查轮询式加锁、锁粒度
JSON 解析	`Json::Reader::*`、`nlohmann::*` 等	避免周期性重复解析，解析一次后缓存
系统调用整体占比偏高	系统调用中的样本超过 20%	汇总项，处理上面几项后会下降

规则阈值由 `--min-pct` 控制（默认 1%），样本较少时可适当调高，避免噪声。

6. 常见问题、扩展与局限

6.1 常见问题

现象	原因	处理
`make` 调用了 `arm-buildroot-linux-gnueabihf-g++` 并报找不到	使用了继承环境变量的旧版 Makefile	换用新版 Makefile，或执行 `make CROSS_COMPILE=`
`perf.data not owned by current user or root`	root 读取 hejr 用户的文件	切换到 hejr 用户，或加 `--force`（`make report` 已默认带上）
`没有解析到任何样本`	输入不是 perf script 输出或折叠栈，或 `--comm` 线程名不对	检查输入文件开头几行和线程名
报告提示 libc 版本不一致	symfs 中的 libc 与板子不同	从板子拷贝 `/lib/arm-linux-gnueabihf/libc-2.30.so` 放到 symfs 相同路径
未解析比例高，只显示 `[libc-2.30.so]`	库被 strip，内部 static 函数无法还原	正常现象；导出函数（gettid、malloc 等）仍能显示
栈底大量 `[perf-N.map]`	回溯越过栈底落到可执行匿名内存	工具已自动去除，不影响结论
业务调用链明显不合理	符号目录与板子程序版本不一致	md5 核对后重建 symfs，重新解析
样本数很少	频率低、时长短或程序空闲	`-F 199`、延长采集，在业务繁忙时采集
root 下路径找不到	`$HOME` 是 `/root`	写完整路径 `/home/hejr/...`

6.2 扩展规则

想要修改的内容	文件与位置
增加或调整问题检测规则	`analyzer.cpp` 中的 `Analyzer::detect()`

想要修改的内容	文件与位置
识别新的库函数、第三方库前缀	`symbol_rules.cpp` 中的 `kLibcNames` 和 `isLibName()`
识别新的内核函数前缀（折叠栈输入）	`symbol_rules.cpp` 中的 `looksLikeKernel()`
增加命令行选项	`options.h` 的 `Options` 和 `options.cpp` 的 `parseOptions()`
报告章节的格式	`analyzer.cpp` 中各 `print*()` 函数

每条检测规则是 `detect()` 中一段独立代码。例如增加"日志输出过多"的检测：

```
// 在 Analyzer::detect() 中、排序 findings 之前加入
{
    set<string> t = {"vfprintf", "__vfprintf_internal", "fwrite", "syslog",
"LogWrite"};
    double w = inclOf(t);              // 包含任一目标函数的样本数
    if (over(w)) {                     // 超过 --min-pct 阈值
        Finding f;
        f.title = "日志输出占用 CPU";
        f.weight = w;
        addCallerDetails(f, t);        // 自动列出业务调用者和所在线程
        f.advice = "降低周期性路径上的日志级别，或改为异步写日志。";
        f.effort = "小";               // 很小 / 小 / 中，影响优先级排序
        findings.push_back(f);
    }
}
```

改完执行 `make` 即可，只会重新编译 `analyzer.cpp`。可复用的辅助函数都是 `Analyzer` 的成员（声明在 `analyzer.h`）：`inclOf()` 统计占比，`appCallers()` 找业务调用者，`byThread()` 按线程统计，`addCallerDetails()` 一次性加入调用者和线程信息。

6.3 局限

- 规则是基于常见模式的启发式判断，不能替代看火焰图，结论需要结合业务代码确认。
- 工具无法判断业务调用链是否合理，符号版本是否匹配仍需人工用 md5 核对。
- CPU 火焰图只反映占用 CPU 的时间，阻塞、等锁、等 IO 的时间不会体现。
- 输入为折叠栈时，内核函数按名字启发式识别；用 perf script 输出作为输入更准确。
- 不同规则之间可能重叠（例如 gettid 也计入系统调用整体占比），占比不能直接相加。

7. 交叉编译与移植到开发板

在现场设备上排查问题时，可以把 perf_analyzer 放到 E3 开发板上运行。使用项目自己的 Buildroot 工具链（`arm-buildroot-linux-gnueabihf-`，gcc 8.3）编译 32 位 ARM 版本，静态链接后不依赖板子上的任何库；E3 是 64 位内核加 32 位用户态，32 位程序可以直接运行。

7.1 设置工具链路径

工具链不在默认 PATH 中，编译前加入（每个终端执行一次，也可写入 `~/.bashrc`）：

```
export PATH=/opt/arm-buildroot-linux-gnueabihf_sdk-buildroot/bin:$PATH
arm-buildroot-linux-gnueabihf-g++ --version
```

7.2 交叉编译

```
cd /home/hejr/test/perfAnalyzer
make clean
make -j4 CROSS_COMPILE=arm-buildroot-linux-gnueabihf- LDFLAGS=-static
arm-buildroot-linux-gnueabihf-strip -o build/perf_analyzer.arm
build/perf_analyzer
file build/perf_analyzer.arm
ls -lh build/perf_analyzer.arm
```

`file` 应显示 `ELF 32-bit LSB executable, ARM, EABI5 ... statically linked, stripped`，大小约 1~2 MB。

- 切换主机版本和交叉编译版本时都必须先 `make clean`，否则会复用另一种架构的目标文件导致链接失败。交叉编译完成后要继续在主机上使用，执行 `make clean && make` 恢复。
- 如果报 `make: arm-buildroot-linux-gnueabihf-g++: No such file or directory`，说明 7.1 的 PATH 没有设置。
- 如果静态链接报 `cannot find -lc` 或 `cannot find -lstdc++`，说明 SDK 没有带静态库，去掉 `LDFLAGS=-static` 改为动态链接即可：Buildroot 工具链的 gcc 8.3 和 glibc 2.30 与板子上的 `libstdc++.so.6.0.25`、`libc-2.30.so` 版本一致，动态链接的程序也能在板子上运行。

7.3 部署到开发板

```
scp build/perf_analyzer.arm root@10.166.64.107:/usr/bin/perf_analyzer
ssh root@10.166.64.107 "chmod +x /usr/bin/perf_analyzer && perf_analyzer --
help"
```

能打印出用法说明即移植成功。板子上同时需要 64 位 perf（`/opt/perf-build64/perf.stripped`，部署为 `/usr/bin/perf`）。

7.4 在开发板上采集和分析

```
echo 0  > /proc/sys/kernel/kptr_restrict
echo -1 > /proc/sys/kernel/perf_event_paranoid

mkdir -p /data/perfwork && cd /data/perfwork
perf record -e cpu-clock -F 199 -m 64 --call-graph fp --proc-map-timeout
10000 \
    -p $(pidof ApplicationServices) -o perf.data -- sleep 120
perf script -i perf.data --show-mmap-events > out.perf 2>/dev/null
perf_analyzer out.perf --duration 120 --addr-list addrs.txt > report.txt
```

- `--show-mmap-events` 必须加，perf_analyzer 要靠其中的内存映射把运行时地址换算成文件内地址；`perf script` 提示 `Failed to open /tmp/perf-N.map` 可以忽略。
- 工作目录选在有足够空间的持久分区（示例中的 `/data`，以 `df -h` 的结果为准），`/tmp` 通常在内存里。
- 采集完成后再执行 `perf script` 和 `perf_analyzer`，避免分析本身的 CPU 占用影响采样结果。
- 板子上没有 Perl，生成火焰图时加 `--folded-out out.folded`，把折叠栈拷回主机用 `flamegraph.pl` 生成。

7.5 板子上的函数名

固件中的程序默认用 `strip --strip-all` 打包，只剩加载段和动态符号表，所以：

部分	板子上直接分析的结果
内核和驱动	函数名（来自 `/proc/kallsyms`）
libc 等系统库	导出函数名，如 `gettid`、`select`、`malloc`（来自 `.dynsym`）
ApplicationServices 和自研库	加 `--addr` / `--addr-list` 时显示"文件名+0x链接地址"，否则只显示库名

可用 `grep -c '\.symtab' /opt/ApplicationServices/ApplicationServices` 检查：输出 `0` 表示被完全 strip。要得到业务函数名，按需选择下面一种方式，三种方式都不需要修改 `--strip-all`。

方式一：板子上记录地址，主机上换算（推荐）

7.4 已经输出了报告 `report.txt` 和地址清单 `addrs.txt`，把地址清单拷回主机换算成函数名即可，完整步骤见第 8 节。

方式二：板子上导出文本，主机上完整分析

```
# 板子
perf script -i perf.data --show-mmap-events 2>/dev/null | gzip > out.perf.gz
# 主机
gunzip -f out.perf.gz
/home/hejr/test/perfAnalyzer/perf_analyzer out.perf --duration 120 \
    --symfs /home/hejr/workspace/facsplat_sw/G4_small/rootfs_arm64-
short/rootfs_partition_unstripped \
    > report.txt
```

得到和主机分析完全相同的报告，包括调用路径和全部规则检测。

方式三：符号副本放在板子上，板子上直接出完整报告

在主机上用 `arm-buildroot-linux-gnueabihf-strip --strip-debug` 从 `rootfs_partition_unstripped` 生成 ApplicationServices 和自研库的副本，按板子上的路径放到 `/data/symfs` 下，然后：

```
perf_analyzer out.perf --duration 120 --symfs /data/symfs > report.txt
```

运行中的程序仍是 `--strip-all` 版本，只有分析时读取副本里的符号表。

版本一致性：三种方式都要求符号来自板子上那个固件对应的同一次编译。确认方法：在主机上对未 strip 文件执行 `arm-buildroot-linux-gnueabihf-strip --strip-all -o /tmp/check <文件>`，`/tmp/check` 的 md5 应与板子上的程序完全一致。调用者一侧记录的是返回地址，极少数会落到相邻函数，判断热点以"自身占比"高的地址为准。

8. 完整流程：开发板生成报告，主机换算地址

本节把"板子上采集并生成报告 → 地址清单拷到主机 → 换算成函数名 → 对照阅读"串成一条完整流程，E3 和 SS 都适用。板子上的程序保持 `--strip-all`，主机上不需要 perf。

8.1 先确认平台

项目	E3	SS
CPU	i.MX8M Mini（Cortex-A53）	AM335x（Cortex-A8）
内核（`uname -m`）	`aarch64`（64 位）	`armv7l`（32 位）
板子上的 perf	64 位版本（`ARCH=arm64` 编译）	32 位版本（`ARCH=arm` 编译，最好用 SS 内核源码）
板子上的 perf_analyzer	32 位 ARM 静态版本（第 7 节），两种平台通用	同左
工程根目录标记文件	`newarm` + `arm64_base_short`（完整版用 `arm64_base_full`）	只有 `newarm`
编译时的平台宏	`-DIMX8MM`	`-DAM335x`
主机符号目录	`rootfs_arm64-short/rootfs_partition_unstripped`	`rootfs_newarm/rootfs_partition_unstripped`
数据目录	有足够空间的持久分区	`/data`（mmcblk1p4，约 2.4 GB 可用）

报告第二部分的"内核架构（推断）"会显示"64 位 ARM"或"32 位 ARM"，可以用来核对。

8.2 编译并升级固件（主机）

```
cd /home/hejr/workspace/facsplat_sw/G4_small
ls newarm arm64_base_short 2>/dev/null       # 确认平台标记
make clean
make build PROFILE=1
```

切换平台或编译选项时必须先 `make clean`，否则不会真正重新编译。`PROFILE=1` 让调用栈可以回溯（实际起作用的是 `-mapcs-frame`），在 SS 上实测根分区只多占用约 0.2 MB。升级固件后记下这次编译对应的符号目录（8.1 表格最后一行），后面换算地址要用。

8.3 确认板子上的程序与符号目录一致

```
# 主机（SS 平台把路径换成 rootfs_newarm）
ROOTFS=/home/hejr/workspace/facsplat_sw/G4_small/rootfs_arm64-short
md5sum $ROOTFS/rootfs_partition/opt/ApplicationServices/ApplicationServices
```

```
# 板子
md5sum /opt/ApplicationServices/ApplicationServices
```

两边必须一致。不一致说明板子上的固件不是这次编译的，换算出来的函数名会错位。

8.4 板子上采集并生成报告和地址清单

```
echo 0  > /proc/sys/kernel/kptr_restrict
echo -1 > /proc/sys/kernel/perf_event_paranoid

mkdir -p /data/perfwork && cd /data/perfwork
perf record -e cpu-clock -F 199 -m 64 --call-graph fp --proc-map-timeout
10000 \
    -p $(pidof ApplicationServices) -o perf.data -- sleep 120
perf script -i perf.data --show-mmap-events > out.perf 2>/dev/null
perf_analyzer out.perf --duration 120 --addr-list addrs.txt > report.txt
ls -l report.txt addrs.txt
```

得到两个文件：

- `report.txt`：完整分析报告。ApplicationServices 和自研库里的函数显示为 `ApplicationServices+0x59641c` 这样的地址；系统库导出函数（`gettid`、`malloc` 等）、自研库对外导出的函数（如 `Json::Reader::parse`、`PersistentStoreService::managerTask()`）和内核函数直接显示名字。
- `addrs.txt`：地址清单，每行是"板子上的文件路径、链接地址、自身样本、累计样本"，只有几 KB。

`--show-mmap-events` 必须加，否则报告第一部分会提示"没有内存映射信息"，地址无法换算。已经有 `perf.data` 时不用重新采集，只需重新执行 `perf script` 和 `perf_analyzer` 两步。

8.5 拷贝到主机

```
mkdir -p /home/hejr/perfwork/ss_0412          # 每次分析一个目录，名字自定
cd /home/hejr/perfwork/ss_0412
scp root@<板子IP>:/data/perfwork/report.txt .
scp root@<板子IP>:/data/perfwork/addrs.txt .
```

8.6 主机上换算成函数名

主机上要用**主机版本**的 perf_analyzer。如果目录里的 `perf_analyzer` 是刚交叉编译的 ARM 版本，先 `make clean && make` 编回主机版本（或把 ARM 版本另存后再编）。

```
# E3
/home/hejr/test/perfAnalyzer/perf_analyzer --resolve-addrs addrs.txt \
    --symfs /home/hejr/workspace/facsplat_sw/G4_small/rootfs_arm64-
short/rootfs_partition_unstripped \
    > funcs.txt

# SS
/home/hejr/test/perfAnalyzer/perf_analyzer --resolve-addrs addrs.txt \
    --symfs
/home/hejr/workspace/facsplat_sw/G4_small/rootfs_newarm/rootfs_partition_unst
ripped \
    > funcs.txt
```

`funcs.txt` 开头会显示"解析成功 N / M 个地址"，正常情况下应接近全部成功。

8.7 对照阅读报告和换算结果

`funcs.txt` 有两张表：

表	内容	用法
按函数汇总	同一函数的多个地址合并后的自身、累计占比和地址数	直接看业务热点函数排行
按地址明细	每个地址对应的"函数+偏移"	把 `report.txt` 里的地址逐个替换成函数名

阅读顺序建议：

1. 在 `report.txt` 第三部分找到最耗 CPU 的线程和它的最热调用路径（全是地址）。
2. 在 `funcs.txt` 的"按地址明细"里查这些地址，得到函数名，还原出完整调用链。例如 SS 上 DataServices 的 `0x59641c → 0xb1b14 → 0xb19f8 → … → 0x140bc8 → Json::Reader::parse`。
3. `report.txt` 第五部分各规则的"调用者"同样按地址明细查找，就能定位到具体是哪个函数在频繁调用 gettid、加锁或解析 JSON。
4. 看"按函数汇总"确认自身占比最高的业务函数，作为优化候选。

只想快速查几个地址时，也可以直接用 addr2line（编译时带了 `-g` 还能显示源文件和行号）：

```
ADDR2LINE=/opt/arm-buildroot-linux-gnueabihf_sdk-buildroot/bin/arm-buildroot-linux-gnueabihf-addr2line
$ADDR2LINE -f -C -e <符号目录>/opt/ApplicationServices/ApplicationServices 0x59641c 0xb1b14 0x140bc8
```

8.8 常见问题

现象	原因	处理
`report.txt` 提示"没有内存映射信息"	`perf script` 没加 `--show-mmap-events`	重新执行 8.4 的 `perf script` 和 `perf_analyzer`
`report.txt` 里业务函数只显示库名	没加 `--addr` / `--addr-list`	加上后重新生成
`funcs.txt` 大量"（未解析）"	符号目录和板子上的固件不是同一次编译，或平台选错（E3/SS 目录混用）	按 8.1 选对目录，按 8.3 核对 md5
`funcs.txt` 提示文件无法打开	`--symfs` 路径错误	确认目录下有 `opt/ApplicationServices/ApplicationServices`
`funcs.txt` 提示"只有动态符号表"	`--symfs` 指向了 strip 过的 `rootfs_partition`	改用 `rootfs_partition_unstripped`
主机上运行报 `Exec format error`	用了交叉编译的 ARM 版本	`make clean && make` 编回主机版本
报告里每个线程的栈底都是同一个地址	线程入口包装函数	报告第三部分已标注并跳过，不是热点
调用者地址换算后落在相邻函数	调用者一侧是返回地址	以自身占比高的地址为准

9. 在主机上生成火焰图

火焰图由 FlameGraph 脚本根据折叠栈（每行一条调用栈加样本数）绘制成 SVG。板子上没有 Perl，所以统一在主机上绘制。原理见《perf_analyzer 工具及火焰图原理》文档。

9.1 准备 FlameGraph（主机，只需一次）

```
git clone https://github.com/brendangregg/FlameGraph.git
/home/hejr/FlameGraph
ls /home/hejr/FlameGraph/flamegraph.pl
```

访问 GitHub 需要代理时，先执行 `git config --global http.proxy http://10.166.64.58:7890`。

9.2 方法一（推荐）：板子导出文本，主机解析符号并画图

函数名完整，E3 和 SS 都适用，主机上不需要 perf。

```
# 板子：已有 perf.data 时直接导出（加 --show-mmap-events）
cd /data/perfwork
perf script -i perf.data --show-mmap-events 2>/dev/null | gzip > out.perf.gz
```

```
# 主机：拷回、解析符号、输出折叠栈、画图
mkdir -p /home/hejr/perfwork/ss_0412 && cd /home/hejr/perfwork/ss_0412
scp root@<板子IP>:/data/perfwork/out.perf.gz . && gunzip -f out.perf.gz

SYMFS=/home/hejr/workspace/facsplat_sw/G4_small/rootfs_newarm/rootfs_partitio
n_unstripped   # E3 用 rootfs_arm64-short
/home/hejr/test/perfAnalyzer/perf_analyzer out.perf --duration 120 --symfs
$SYMFS \
    --folded-out out.folded > report.txt
/home/hejr/FlameGraph/flamegraph.pl --title "ApplicationServices" out.folded
> flame.svg
```

一次得到带完整函数名的 `report.txt` 和 `flame.svg`。符号目录必须和板子上的固件是同一次编译（第 8.3 节 md5 核对）。

9.3 方法二：板子直接输出折叠栈（地址形式）

不想拷贝 `out.perf` 时，在板子上生成报告时顺便输出折叠栈，只拷这一个小文件：

```
# 板子
perf_analyzer out.perf --duration 120 --addr-list addrs.txt --folded-out
out.folded > report.txt
```

```
# 主机
scp root@<板子IP>:/data/perfwork/out.folded .
/home/hejr/FlameGraph/flamegraph.pl --title "ApplicationServices（地址）"
out.folded > flame_addr.svg
```

火焰图里业务函数显示为 `ApplicationServices+0x59641c` 这样的地址，结构和比例是准确的；配合第 8 节换算出的 `funcs.txt` 查地址对应的函数名。需要带函数名的火焰图时用方法一。

9.4 方法三：主机上用 perf 解析 perf.data

主机有 perf、并已准备好符号目录和 `kallsyms` 时，可以用 Makefile 一键生成（见第 3 节）：

```
make -C /home/hejr/test/perfAnalyzer report DATA=<perf.data> KALLSYMS=
<kallsyms> \
    SYMFS=<符号目录> FLAMEGRAPH=/home/hejr/FlameGraph DURATION=120
```

9.5 常用变体

需求	做法
只看一个线程	生成折叠栈时加 `--comm DataServices`
内核函数用不同颜色	生成折叠栈时加 `--mark-kernel`，画图时加 `--color=java`（内核橙色、用户态绿色）
冰柱图（从叶子往根看，适合找共同的底层热点函数）	`flamegraph.pl --reverse --inverted out.folded > icicle.svg`
对比优化前后	`difffolded.pl before.folded after.folded | flamegraph.pl > diff.svg`，红色变多、蓝色变少
图太窄、名字显示不全	`flamegraph.pl --width 1800 ...`
保留栈底的 `[perf-N.map]` 帧	生成折叠栈时加 `--keep-root-anomalies`（默认已去掉）

9.6 查看火焰图

用浏览器打开 `flame.svg`（Chrome、Firefox、Edge 都可以）：

- 横向宽度代表占用 CPU 的比例，越宽越热；从下往上是调用关系，最上面一层是正在执行的函数。
- 鼠标悬停显示函数名、样本数和占比；点击某个函数放大到以它为根，点左上角 Reset Zoom 还原。
- 右上角 Search 输入函数名（支持正则，如 `Json|gettid`），匹配的函数高亮显示，右下角显示合计占比。
- 横向顺序按函数名字母排序，不代表时间先后。